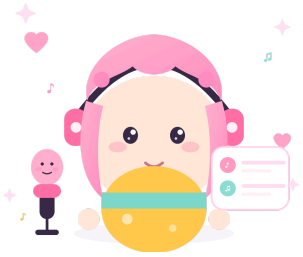
Taller de Lenguajes 2 — Trabajo Final Integrador

Sistema de Gestión de una Plataforma de Streaming de Audio

Entregable 2



Introducción

A partir del diseño e implementación del entregable 1 en este nuevo entregable se incorporará funcionalidad necesaria para persistir la información en una base de datos.

Consigna de la Entrega 2

Este entrega consta de 3 partes:

1. Configuración de la base de datos y creación de tablas

- Crear tablas para cada clase que se deba persistir. A modo de ejemplo, el siguiente fragmento de código sirve de base para escribir el código que crea las tablas.
- Configurar una base de datos SQLite, para ello agregar la librería de SQLite (archivo .jar) dentro del proyecto Java.
    - Agregar el archivo sqlite al .gitignore para no subir la base de datos al repositorio.

  Descargar el .jar de la siguiente URL:
  https://repo1.maven.org/maven2/org/xerial/sqlite-jdbc/3.53.4.0/sqlite-jdbc-3.53.4.0.jar

  Herramienta para visualizar las tablas de la base de datos:
  DBeaver: https://sqliteviewer.app/

2. Implementación del patrón DAO

- Implementar una clase DAO para cada tabla creada en el punto anterior.
- Cada método de las clases DAO debe estar correctamente documentado con comentarios Javadoc en el que se describe qué hace el método, qué recibe como parámetro y qué retorna.

3. Organización del código de acuerdo al patrón de diseño MVC

Paquetes para separar el modelo de las funcionalidades, y paquete para vista (entrada y salida)

4. **Investigar sobre persistencia de archivos**

   - Investigar sobre las distintas alternativas para persistir los distintos contenidos de audio en la base de datos.
   - Escribir en el archivo README.md del repositorio la propuesta de implementación.

5. **Para los grupos de 3 integrantes**

Implementar en forma completa al menos las siguientes funcionalidades:

a) Funcionalidades generales de la plataforma:

- Agregar usuarios a la plataforma: al menos 2 de cada tipo de usuario.

  Realizar las siguientes validaciones:

  - No se permite el ingreso de números en los campos nombres y apellido.
  - Verificar que el DNI/email sea único.
  - Ingresar todos los datos.

  La plataforma debe informar los datos erróneos mostrando la salida por consola.

- Publicar contenidos de diferentes géneros al catálogo de la plataforma (usuario creador): al menos un contenido de cada tipo.
  - Persistir el archivo de audio en la base de datos de acuerdo a la investigación realizada en el punto anterior.
  - Realizar las siguientes validaciones:
    - Los álbumes contienen únicamente música.
    - Los géneros deben ser válidos.
    - Ingresar todos los datos.
    - Verificar la inexistencia de datos duplicados.
- Filtrar contenido por género: mostrar por consola el resultado del filtro.
- Consultar el contenido con más reproducciones del catálogo: mostrar por consola el resultado de la consulta.

b) Funcionalidades específicas de un usuario: es necesario validar que sea un usuario de la plataforma mediante un login (usuario/contraseña)

- Simular la reproducción de un contenido:
  - Mostrar por consola los datos del archivo a reproducir (título, autor, género, etc)
  - Actualizar las estadísticas de reproducción del usuario y mostrarla por consola.

- Agregar un contenido al playlist de un usuario.

6. Para los grupos de 2 integrantes

Implementar en forma completa al menos las siguientes funcionalidades generales de la plataforma:

- Agregar usuarios a la plataforma: al menos 2 de cada tipo de usuario.

  Realizar las siguientes validaciones:

  - No se permite el ingreso de números en los campos nombres y apellido.
  - Verificar que el DNI/email sea único.
  - Ingresar todos los datos.

  La plataforma debe informar los datos erróneos mostrando la salida por consola.

- Publicar contenidos de diferentes géneros al catálogo de la plataforma (usuario creador): al menos un contenido de cada tipo.

  Realizar las siguientes validaciones:

  - Los álbumes contienen únicamente música.
  - Los géneros deben ser válidos.
  - Ingresar todos los datos.
  - Verificar la inexistencia de datos duplicados.

- Filtrar contenido por género: mostrar por consola el resultado del filtro.
- Consultar el contenido con más reproducciones del catálogo: mostrar por consola el resultado de la consulta.

¿Qué se entrega?

- Proyecto Java actualizado con las nuevas clases DAO y las adecuaciones del código y del UML de acuerdo a las correcciones del entregable 1.
- Subir el proyecto a la rama main del repositorio. Deben estar implementadas todas las funcionalidades pedidas de acuerdo a la cantidad de integrantes del grupo.

Temas a evaluar

- Patrón DAO.
- Buenas prácticas de programación.
- Organización del proyecto java en paquetes respetando el patrón MVC.
- Documentación de código con Javadoc.
- Correcto uso de la herramienta Git.
  - Uso de ramas y al menos un commit significativo por de cada integrante del grupo

- Uso de archivo .gitignore para no subir los archivos de audio ni la base de datos al repositorio

Modalidad de entrega y fechas

- Fecha: 16/11/2026
- Forma de entrega: subir el proyecto Java a la rama main del repositorio GitLab del grupo.

```
/**
       * Este método se encarga de la creación de las tablas donde se
      * almacenara la información de los objetos.
      * Una vez establecida una conexión, debería ser lo próximo a ser ejecutado.
       * @param connection objeto conexión a la base de datos SQLite
       * @throws SQLException
       */

private void creacionDeTablasEnBD(Connection connection) throws SQLException {
    String[] ddl = {
        """
        CREATE TABLE IF NOT EXISTS USUARIO (
            ID             INTEGER PRIMARY KEY AUTOINCREMENT,
            NOMBRE         TEXT NOT NULL,
            APELLIDO       TEXT NOT NULL,
            DNI            INTEGER NOT NULL UNIQUE,
            EMAIL          TEXT NOT NULL UNIQUE,
            NOMBRE_USUARIO TEXT NOT NULL UNIQUE
        )""",
        """
        CREATE TABLE IF NOT EXISTS CONTENIDO_AUDIO (
            ID                INTEGER PRIMARY KEY AUTOINCREMENT,
            TITULO            TEXT NOT NULL,
            DURACION          INTEGER NOT NULL CHECK (DURACION > 0),
            FECHA_PUBLICACION TEXT NOT NULL,
            ID_USUARIO        INTEGER NOT NULL,
            CONSTRAINT CONTENIDO_AUDIO_USUARIO_FK
                FOREIGN KEY (ID_USUARIO) REFERENCES USUARIO(ID)
        )""",
        """
        CREATE TABLE IF NOT EXISTS PLAYLIST (
            ID             INTEGER PRIMARY KEY AUTOINCREMENT,
            NOMBRE         TEXT NOT NULL,
            DESCRIPCION    TEXT,
            FECHA_CREACION TEXT NOT NULL,
            ID_USUARIO     INTEGER NOT NULL,
            CONSTRAINT PLAYLIST_USUARIO_FK
                FOREIGN KEY (ID_USUARIO) REFERENCES USUARIO(ID) ON DELETE CASCADE
        )""",
        """
```

```
        CREATE TABLE IF NOT EXISTS PLAYLIST_CONTENIDO (
            ID_PLAYLIST        INTEGER NOT NULL,
            ID_CONTENIDO_AUDIO INTEGER NOT NULL,
            POSICION           INTEGER NOT NULL,
            PRIMARY KEY (ID_PLAYLIST, ID_CONTENIDO_AUDIO),
            CONSTRAINT PLAYLIST_CONTENIDO_PLAYLIST_FK
                FOREIGN KEY (ID_PLAYLIST) REFERENCES PLAYLIST(ID) ON DELETE
CASCADE,
            CONSTRAINT PLAYLIST_CONTENIDO_AUDIO_FK
                FOREIGN KEY (ID_CONTENIDO_AUDIO) REFERENCES CONTENIDO_AUDIO(ID) ON
DELETE CASCADE
        )""",
        """
        CREATE INDEX IF NOT EXISTS IDX_PLAYLIST_CONTENIDO_AUDIO
            ON PLAYLIST_CONTENIDO (ID_CONTENIDO_AUDIO)
        """
    };

    boolean autoCommit = connection.getAutoCommit();
    connection.setAutoCommit(false);
    try (Statement stmt = connection.createStatement()) {
        for (String sql : ddl) {
            stmt.executeUpdate(sql);
        }
        connection.commit();
    } catch (SQLException e) {
        connection.rollback();
        throw e;
    } finally {
        connection.setAutoCommit(autoCommit);
    }
}
```

Fragmento de código Java sobre creación de tablas de una base de datos relacional